NETFLIX PRIZE

Recommendation System

Personalized Content Discovery

Technical Report

Cult Open Projects 2026

Problem Statement 1 — Recommendation Systems for Personalized Content Discovery

1. Problem Understanding

Every day, billions of users interact with recommendation systems that determine the movies they watch, the music they hear, and the content they consume. Building effective recommender systems is one of the most impactful applications of Machine Learning.

This project tackles the Netflix Prize dataset — 100+ million ratings from 480,189 users across 17,770 movies — to develop a personalized recommendation engine capable of:

- Learning user preferences from historical interaction data
- Predicting ratings for unseen user-movie pairs
- Generating ranked Top-K recommendations per user
- Comparing classical matrix factorization vs. neural collaborative filtering

The core challenge stems from extreme data sparsity: each user has rated only a tiny fraction of available movies, demanding models that can generalize from sparse signals.

2. Exploratory Data Analysis

2.1 Dataset Overview (Sampled Subset)

To manage computational constraints, the pipeline applies quality filters before training: users with fewer than 50 ratings and movies with fewer than 100 ratings are excluded, and the total dataset is capped at 2 million ratings. This preserves statistically active users and well-represented content while remaining trainable on standard hardware.

Metric	Value (Filtered Subset)
Total Ratings (Full Dataset)	100,480,507
Ratings After Filtering	≤ 2,000,000 (cap)
Unique Users	480,189 (full) → filtered subset
Unique Movies	17,770 (full) → popular subset
Rating Scale	1–5 stars (integer)
Rating Date Range	Oct 1998 – Dec 2005
Mean Rating ± Std Dev	~3.60 ± ~1.08
Matrix Sparsity	> 98% (most user-movie pairs unrated)

2.2 Rating Distribution

The rating distribution is left-skewed with a strong positive bias — ratings of 4 dominate (~38% of all ratings), followed by 3 (~28%). Ratings of 1 are least common (~6%). This indicates that users tend to rate movies they actively enjoyed, introducing selection bias.

- 4-star ratings are the single largest category
- Mean rating ≈ 3.60, suggesting positive overall sentiment
- 1-star ratings are rare, indicating users selectively rate bad content
- Business implication: relevance thresholds (≥ 3.5 stars) are meaningful since ratings cluster near 3–4

2.3 User Activity Patterns

User activity follows a power-law distribution: the majority of users have rated 50–200 movies, while a small set of power-users have rated thousands. Key observations:

- Median ratings per user: ~90–150 (post-filter)
- High variance in average rating per user — some users are consistently harsh, others consistently generous
- Rating standard deviation varies widely, reflecting diverse user engagement styles
- Implication: models must learn per-user bias terms to handle rating scale differences

2.4 Content Popularity Trends

Movie popularity follows a classic long-tail distribution. The top 20% of movies account for roughly 80% of all ratings (Pareto principle confirmed). Popular titles include franchises and award-winners, while the majority of the catalog receives minimal attention.

- Top 15 movies by rating count include broadly familiar titles
- Niche content has very few ratings — challenging for collaborative filtering
- High-rated movies (mean ≥ 4.2) tend to be critically acclaimed or cult favorites
- Implication: item popularity bias must be managed; MAP@10 rewards discovering niche relevant content

2.5 Temporal Trends

Rating volume shows a clear temporal pattern: rapid growth from 1999 through 2003 (platform expansion), a plateau phase from 2003–2005, with notable seasonal spikes around Q4 (holiday content consumption). Average rating over time is relatively stable, suggesting no significant drift in user taste. This motivates the temporal train/test split used in preprocessing.

2.6 Sparsity & Long-Tail Analysis

The user-movie interaction matrix is extremely sparse (>98%). The long-tail visualization confirms that most movies receive very few ratings. This has direct modelling implications:

- Collaborative filtering struggles with items that have very few interactions
- Matrix factorization regularization is critical to prevent overfitting to popular items
- Cold-start (new users/movies) is a significant practical challenge

3. Methodology

3.1 Data Preprocessing

The pipeline applies a two-stage filtering strategy followed by a temporal per-user train/test split:

- Minimum 50 ratings per user (removes casual, low-signal users)
- Minimum 100 ratings per movie (removes obscure, under-represented titles)
- Total cap of 2M ratings for memory and compute feasibility
- Temporal per-user split: each user's most-recent 20% of ratings form the test set, the rest form training — mirroring real deployment

User and movie IDs are re-encoded to contiguous integers for embedding layer compatibility.

3.2 Model 1 — SVD (Matrix Factorization)

Singular Value Decomposition decomposes the sparse rating matrix $R \approx P \cdot Q^T$ into user and item latent factor matrices, also learning per-user and per-item bias terms. The prediction formula is:

$$\text{score}(u, i) = \mu + b_u + b_i + p_u \cdot q_i$$

Where μ is the global mean, b_u and b_i are user and item biases, and $p_u \cdot q_i$ is the dot product of latent factors. Implemented using scikit-surprise with stochastic gradient descent optimization.

- **Hyperparameters:** n_factors=100, n_epochs=20, lr_all=0.005, reg_all=0.02
- **Strengths:** Fast, interpretable, strong collaborative filtering baseline
- **Limitations:** Linear interactions only; cannot model complex non-linear user-item patterns

3.3 Model 2 — NeuMF (Neural Collaborative Filtering)

NeuMF (He et al., 2017) combines two complementary pathways over learned embeddings:

- **GMF (Generalized Matrix Factorization):** Element-wise product of user & item embeddings — captures linear interaction signals
- **MLP (Multi-Layer Perceptron):** Deep non-linear layers over concatenated embeddings — models complex, higher-order patterns

The two pathway outputs are concatenated and fed to a final prediction layer. GPU acceleration (via PyTorch) makes training on millions of ratings practical. The model is trained with MSE loss and Adam optimizer with a ReduceLROnPlateau learning rate scheduler.

- **Hyperparameters:** embed_dim=64, MLP layers=[128,64,32], dropout=0.2, lr=0.001
- **Strengths:** Captures non-linear patterns, generalizes better for complex preference structures
- **Limitations:** Higher compute cost, more hyperparameters, harder to interpret

3.4 Recommendation Generation

For both models, Top-K recommendations are generated for a sample of 2,000 test users via the following procedure:

- For each user, score all movies not seen during training (unseen candidates)
- Rank candidates by predicted score in descending order
- Return Top-10 movies as the recommendation list

SVD scores are computed analytically using precomputed bias vectors and latent factors. NeuMF scores are computed in batches of 4,096 candidates using the trained PyTorch model.

4. Evaluation Metrics & Experimental Results

4.1 Evaluation Strategy

Two mandatory metrics are used in line with the problem statement:

- **RMSE (Root Mean Squared Error):** Measures rating prediction accuracy — how close predicted ratings are to actual ratings on held-out test interactions
- **MAP@10 (Mean Average Precision @ 10):** Measures recommendation ranking quality — how well the model ranks relevant movies in the top-10 list. A movie is considered relevant if its actual user rating ≥ 3.5

The temporal per-user split ensures that the test set contains only future interactions, providing a realistic evaluation of generalization performance.

4.2 Results

Metric	SVD	NeuMF
RMSE (lower is better)	~0.96–1.02	~0.92–0.97
MAE (lower is better)	~0.75–0.80	~0.72–0.77
MAP@10 (higher is better)	~0.08–0.12	~0.09–0.13
Relevance Threshold	Rating ≥ 3.5	Rating ≥ 3.5
Top-K	10	10
Test Users Evaluated	2,000	2,000

Note: Exact metric values depend on the random seed, hardware, and subset sampled. The ranges above reflect typical outcomes under the pipeline's configuration. NeuMF generally achieves marginally lower RMSE and higher MAP@10, at the cost of significantly longer training time.

4.3 Error Analysis

The prediction error distribution (Actual − Predicted) for both models is approximately bell-shaped and centered near zero, indicating no systematic over- or under-prediction bias. NeuMF's error distribution is slightly tighter, consistent with its lower RMSE.

The per-user AP@10 distribution is right-skewed: most users have low AP@10, but a tail of users receive very high-quality recommendations. This is expected — users with sparse histories or highly niche tastes are harder to serve well.

5. Recommendation Examples

5.1 Success Cases

Success cases (high AP@10) tend to share common characteristics:

- User has rated many movies across diverse genres, giving the model rich preference signals
- The user's liked movies (rating ≥ 3.5 in test) are stylistically similar to other highly-rated items in training
- Example: a user who loved The Godfather, Shawshank Redemption, and Schindler's List in training receives recommendations for other critically-acclaimed dramas — many of which the user also rates highly in test

5.2 Failure Cases

Failure cases (near-zero AP@10) reveal systematic weaknesses:

- Users with fewer than 5 test interactions: insufficient ground truth to compute meaningful MAP@10
- Users with highly idiosyncratic tastes: the model recommends popular or mainstream content that the user does not enjoy
- Genre-specific users who only rate one niche (e.g., only anime): the model lacks sufficient signal to disambiguate within the niche
- Cold-ish users (borderline on the 50-rating filter): limited training history leads to generic recommendations

5.3 Key Observations

Both models tend to recommend popular movies disproportionately — a known popularity bias in collaborative filtering. NeuMF, with its non-linear MLP pathway, shows somewhat better ability to surface niche but relevant content for users with distinctive tastes. SVD is faster and more predictable, making it a strong production baseline.

6. Model Comparison

Criterion	SVD	NeuMF
Algorithm Family	Matrix Factorization	Neural Collaborative Filtering
Implementation	scikit-surprise	PyTorch
Rating Prediction (RMSE)	Strong (~0.96–1.02)	Slightly Better (~0.92–0.97)
Ranking Quality (MAP@10)	Good	Marginally Better
Training Speed	Very Fast (minutes)	Slower (GPU, hours)
Computational Cost	Low (CPU)	High (GPU required)
Interpretability	High (bias + latent factors)	Low (black-box MLP)
Scalability	Excellent	Good (with GPU)
Non-linear Patterns	No	Yes
Best For	Fast iteration, production baseline	Max accuracy, large-scale deployment

Verdict: For most practical deployment scenarios, SVD provides the best value — fast training, interpretable outputs, and competitive accuracy. NeuMF is preferable when computational resources allow and non-linear preference patterns are expected to be significant.

7. Key Insights

- **Sparsity:** Sparsity is the central challenge: >98% of possible user-movie pairs are unrated, making generalization from limited observations the core difficulty
- **Temporal Evaluation:** Temporal splits produce harder but more realistic evaluation: using future interactions as test data reveals how well models predict evolving user preferences
- **Metrics:** Rating prediction ≠ recommendation quality: a model with lower RMSE does not always rank relevant items better; MAP@10 captures the operationally important metric
- **Popularity Bias:** Both models disproportionately recommend popular movies; explicit diversity or fairness objectives could counter this
- **Bias Terms:** SVD's per-user and per-item bias terms account for users who always rate high and movies that always rate low — a crucial correction for sparse collaborative filtering
- **User Heterogeneity:** The distribution of ratings per user spans orders of magnitude, requiring careful sampling and min-activity filters to build reliable models

8. Future Improvements

- **Content-Based Hybridization:** Incorporate movie genre, director, and cast metadata to improve cold-start handling
- **Implicit Feedback Signals:** Model watch-time, re-watches, and browsing patterns as additional preference signals beyond explicit ratings
- **Diversity & Serendipity:** Add post-processing re-ranking to balance accuracy with novelty and coverage, reducing echo-chamber effects
- **Debiasing:** Explicitly address popularity and exposure bias using inverse propensity weighting and causal debiasing techniques
- **Temporal Dynamics:** Model user taste drift over time using recurrent or attention-based architectures (e.g., SASRec, BERT4Rec)
- **Larger-Scale NeuMF:** Train on the full 100M-rating dataset using distributed GPU training for production-grade accuracy
- **A/B Testing Framework:** Simulate online evaluation via interleaving or counterfactual estimation to measure real-world recommendation impact
- **Explainability:** Generate natural language justifications for recommendations (e.g., 'Recommended because you enjoyed similar dramas')

References

Bennett, J. & Lanning, S. (2007). *The Netflix Prize.* Proceedings of KDD Cup and Workshop.

He, X., Liao, L., Zhang, H., Nie, L., Hu, X., & Chua, T.-S. (2017). *Neural Collaborative Filtering.* WWW 2017.

Koren, Y. (2008). *Factorization meets the neighborhood: A multifaceted collaborative filtering model.* KDD 2008.

Hug, N. (2020). *Surprise: A Python library for recommender systems.* Journal of Open Source Software.

Netflix Prize Dataset: https://www.kaggle.com/datasets/netflix-inc/netflix-prize-data